The stage ladder

One cohort, driven by hand: **one statement per rung**, from created to closed. Nothing is on a clock, so nothing expires while you work. Every rung says what both dashboards will read afterwards, and what you have to click before the next one.

RUNG		WRITES	CONSUMER	PARTNER	PARTNER CAN BID
0	create	one INSERT	forming	planned	no button
1	announce	announce_at	locked	announced	no button
2	auction	kind and the flag	locked	announced	no button
3	bidding	bidding_opens_at	bidding	open	**BID WINDOW**
4	close	bidding_closes_at	offers	offers_out	no button
5	offers	offers_at	offers	offers_out	no button
6	decide	decision_at	confirm	decided	no button
7	switch	switch_window_at	switching	decided	no button
8	reconcile	reconcile_at	done	decided	no button
9	done	kind and the flag	done	decided	no button

Left to right: the rung, what its statement writes, the stage the consumer dashboard shows, the stage the partner desk shows, and whether a partner can bid. Exactly one rung opens the bid window, and exactly one closes it.

Print this one

The ladder is meant to sit beside the ZCQL console while you work through it.

Generated by

`node scripts/cohort.mjs step <ID> --list`, which fills in every timestamp for you.

Companions

cohort-launch-guide.pdf for creating cohorts, docs/COHORT_RUNBOOK.md for the long path.

BEFORE YOU START

Four things that make the ladder work

All four are consequences of one design decision: a stage is **derived from the dates that exist**, never stored and never counted.

1. A date you have not written cannot expire

Every rung stamps one date and leaves the later ones NULL. So a bid window opened at rung 3 stays open until rung 4 closes it: `requireBiddingOpen` only refuses past `bidding_closes_at`, and until rung 4 there is no such date. This is the whole reason to drive by hand instead of setting a timed calendar: no rung has a deadline you did not ask for.

2. The ladder is resumable, because the row is the state

Nothing tracks where you are. The dates on the row are where you are, so you can stop for a day, come back, and run the next rung. Running one twice is harmless: it restamps the same date with a later minute.

3. Rung 0 is the only chance to join

Joining is open on `planned`, `waitlist` and `forming`. Rung 2 makes the cohort an auction and **locks joining permanently**. Worse, the member rail only advances by itself for a member who has joined that cohort: without a join it polls every 120 seconds instead of every 15. **Join at rung 0 or the consumer half of the test is not watchable.**

4. Two habits, every rung

One statement per submission. The ZCQL console takes one; a block of nine is a syntax error. **Then wait 60 seconds**, because the catalog is memoized and hand-written ZCQL cannot invalidate it. And **reload /partner** every time: the console fetches campaigns once at boot and never polls. The member dashboard is the only side that moves on its own.

The vocabulary, so the two columns are readable

CONSUMER STAGES	PARTNER STAGES
`forming, locked, bidding, offers, confirm, switching, done`	`planned, announced, open, closing, offers_out, decided`

Seven for the household, six for the partner, both derived from the same seven dates. They are different vocabularies because the two sides are watching different films: a partner is finished at `decided`, a household still has an offer to confirm and an install to sit through.

THE LADDER

Nine rungs, nine statements

Every rung below is a **pair**, and the order matters.

Run the terminal command first. Paste the statement second

1. The terminal command writes nothing. It prints the statement with the UTC timestamp already filled in, validates the id against the same constants the server uses, and predicts what both dashboards will read afterwards by calling the server's own stage functions. That prediction is the point: it is how you tell "the rung did not work" apart from "the rung worked, and this is what it looks like".

2. The dark block is the write, and the only one. What is printed here carries `<NOW>` and `<ID>` because a document cannot know either, so it is the shape rather than something to paste. The command in step 1 is what turns the shape into a statement you can paste.

Skipping step 1 is legal and costs you three things: you type the UTC minute by hand in `'YYYY-MM-DD HH:MM:SS'`, which is neither ISO-8601 nor your local time; nothing checks the value, and a mistyped `kind` reads back as `planned` in silence; and you find out what the rung did by looking, rather than by knowing first.

0 Create the cohort, with no calendar at all

Not a rung: the row every rung after this one updates.

1 · TERMINAL · PRINTS THE STATEMENT. WRITES NOTHING

```
node scripts/cohort.mjs new <ID> --region "<REGION>"
```

2 · ZCQL · PASTE THIS. THE ONLY THING THAT WRITES

```
INSERT INTO campaigns (campaign_id, region, sub, kind, target, seed_members,
  seed_households, bidding_open, sort_order, updated_by, updated_at)
VALUES ('<ID>', '<REGION>', '<SUB>', 'forming', 100, 0, 0, false, <SORT>, 'manual',
'<NOW>');
```

AFTER IT	STAGE	WHAT THE SURFACE SHOWS
Consumer	`forming`	On the row as **Open to join**, and the featured card while nothing else is joinable.
Partner	`planned`	Under **Coming cohorts**, headed "Planned in your coverage", status **Still forming**. Every approved partner sees it. No action column in that table at all.

You do: Join it, as a test member, on /dashboard. This is the only rung where that is possible, and the member rail will not advance for anyone who has not joined.

1 The brief is fixed and joining shuts

Stamps `announce_at` with the minute you run it, and leaves every later date NULL.

1 · TERMINAL · PRINTS THE STATEMENT. WRITES NOTHING

```
node scripts/cohort.mjs step <ID> --to announce
```

2 · ZCQL · PASTE THIS. THE ONLY THING THAT WRITES

```
UPDATE campaigns SET announce_at = '<NOW>', updated_at = '<NOW>'
WHERE campaign_id = '<ID>';
```

AFTER IT	STAGE	WHAT THE SURFACE SHOWS
Consumer	`locked`	The rail moves to **Locked** on its next poll, 15 seconds.
Partner	`announced`	Still under **Coming cohorts**, status now **Announced**. Nothing to bid on: it is not in the Open auctions table yet.

You do: Nothing to click.

2 It becomes an auction, and the window opens

No date. This one moves the lifecycle, not the calendar.

1 · TERMINAL · PRINTS THE STATEMENT. WRITES NOTHING

```
node scripts/cohort.mjs step <ID> --to auction
```

2 · ZCQL · PASTE THIS. THE ONLY THING THAT WRITES

```
UPDATE campaigns SET kind = 'auction', bidding_open = true, updated_at = '<NOW>'
WHERE campaign_id = '<ID>';
```

AFTER IT	STAGE	WHAT THE SURFACE SHOWS
Consumer	`locked`	Unchanged, but **joining is now permanently closed**. There is no route back.
Partner	`announced`	Still under **Coming cohorts** as **Announced**. The flag alone is not a stage, so the row has not moved tables.

You do: Reload /partner if you want to see that the row has not changed yet. That is the point of the next rung.

3 Sealed bidding is live

Stamps `bidding_opens_at` with the minute you run it, and leaves every later date NULL.

1 · TERMINAL · PRINTS THE STATEMENT. WRITES NOTHING

```
node scripts/cohort.mjs step <ID> --to bidding
```

2 · ZCQL · PASTE THIS. THE ONLY THING THAT WRITES

```
UPDATE campaigns SET bidding_opens_at = '<NOW>', updated_at = '<NOW>'
WHERE campaign_id = '<ID>';
```

AFTER IT	STAGE	WHAT THE SURFACE SHOWS
Consumer	`bidding`	The rail reads **Bidding**. No price, no count, nothing about any bid.
Partner	`open`	Moves into **Open auctions**, reads **Open**, and draws **Review and bid** where coverage is active, or a "Verifies with <REGION> coverage" lock where it is not. No close date yet, so the window does not expire while you work.

You do: Reload /partner and place the bid. Fill the ticket in the console; do not hand-roll the body.

4 Bids close, and the seal opens

Stamps `bidding_closes_at` with the minute you run it, and leaves every later date NULL.

1 · TERMINAL · PRINTS THE STATEMENT. WRITES NOTHING

```
node scripts/cohort.mjs step <ID> --to close
```

2 · ZCQL · PASTE THIS. THE ONLY THING THAT WRITES

```
UPDATE campaigns SET bidding_closes_at = '<NOW>', updated_at = '<NOW>'
WHERE campaign_id = '<ID>';
```

AFTER IT	STAGE	WHAT THE SURFACE SHOWS
Consumer	`offers`	The winning offer becomes readable. Before this rung the answer was `sealed: true` and nothing else.
Partner	`offers_out`	Reads **Offers out**. **Every further bid is refused from here**, whatever the flag says.

You do: Nothing to click, but this is the rung to check the offer panel on /dashboard.

5 The offer has reached the household

Stamps `offers_at` with the minute you run it, and leaves every later date NULL.

1 · TERMINAL · PRINTS THE STATEMENT. WRITES NOTHING

```
node scripts/cohort.mjs step <ID> --to offers
```

2 · ZCQL · PASTE THIS. THE ONLY THING THAT WRITES

```
UPDATE campaigns SET offers_at = '<NOW>', updated_at = '<NOW>'
WHERE campaign_id = '<ID>';
```

AFTER IT	STAGE	WHAT THE SURFACE SHOWS
Consumer	`offers`	Same stage, now dated. The rail says the offer is in rather than inferring it.
Partner	`offers_out`	Unchanged.

You do: Accept the offer on /dashboard. That is the act that creates the switch order and the only thing that puts an address in front of a partner.

6 Confirmations lock

Stamps `decision_at` with the minute you run it, and leaves every later date NULL.

1 · TERMINAL · PRINTS THE STATEMENT. WRITES NOTHING

```
node scripts/cohort.mjs step <ID> --to decide
```

2 · ZCQL · PASTE THIS. THE ONLY THING THAT WRITES

```
UPDATE campaigns SET decision_at = '<NOW>', updated_at = '<NOW>'
WHERE campaign_id = '<ID>';
```

AFTER IT	STAGE	WHAT THE SURFACE SHOWS
Consumer	`confirm`	The rail reads **Confirm**.
Partner	`decided`	Reads **Decided**, and stays there for every rung after this one.

You do: Nothing to click.

7 Installs and transfers run

Stamps `switch_window_at` with the minute you run it, and leaves every later date NULL.

1 · TERMINAL · PRINTS THE STATEMENT. WRITES NOTHING

```
node scripts/cohort.mjs step <ID> --to switch
```

2 · ZCQL · PASTE THIS. THE ONLY THING THAT WRITES

```
UPDATE campaigns SET switch_window_at = '<NOW>', updated_at = '<NOW>'
WHERE campaign_id = '<ID>';
```

AFTER IT	STAGE	WHAT THE SURFACE SHOWS
Consumer	`switching`	The rail reads **Switching**.
Partner	`decided`	Unchanged. The work is on the delivery view, not the desk.

You do: Mark the activation on /partner. Only an activation with a clean line test creates a fee: not a confirmation, not an acceptance, not a booking.

8 Final counts settle

Stamps `reconcile_at` with the minute you run it, and leaves every later date NULL.

1 · TERMINAL · PRINTS THE STATEMENT. WRITES NOTHING

```
node scripts/cohort.mjs step <ID> --to reconcile
```

2 · ZCQL · PASTE THIS. THE ONLY THING THAT WRITES

```
UPDATE campaigns SET reconcile_at = '<NOW>', updated_at = '<NOW>'
WHERE campaign_id = '<ID>';
```

AFTER IT	STAGE	WHAT THE SURFACE SHOWS
Consumer	`done`	The rail reads **Done**. Seven stages, all of them from dates.
Partner	`decided`	Unchanged.

You do: Nothing to click.

9 The cohort is closed

No date. This one moves the lifecycle, not the calendar.

1 · TERMINAL · PRINTS THE STATEMENT. WRITES NOTHING

```
node scripts/cohort.mjs step <ID> --to done
```

2 · ZCQL · PASTE THIS. THE ONLY THING THAT WRITES

```
UPDATE campaigns SET kind = 'closed', bidding_open = false, updated_at = '<NOW>'
WHERE campaign_id = '<ID>';
```

AFTER IT	STAGE	WHAT THE SURFACE SHOWS
Consumer	`done`	Results only.
Partner	`decided`	Closed. Leaving auction always closes the window, which is what this statement does explicitly.

You do: Optional. `move <ID> --to archived` later takes it off every non-admin surface.

AROUND THE LADDER

What the statements cannot do

Four of the rungs need a person in a browser, and two of the four are the interesting ones to test, because they are the writes that create records.

RUNG	YOU DO	WHY IT IS NOT A STATEMENT
0	**Join the cohort** on /dashboard	Writes `campaign_members` through `POST /campaigns/join`. Doing it by hand would skip the flattened membership key the route builds, and the count on both dashboards is that table.
3	**Place the bid** on /partner	`lib/bids.js` validates tiers, technologies, speeds, sticker and effective prices, the guarantee term, equipment and the commitment cap, then writes the sealed revision and a receipt. Never hand-roll the body.
5	**Accept the offer** on /dashboard	The one act that puts a household address in front of a partner, and the only thing that creates a switch order.
7	**Mark the activation** on /partner	The only event that creates a billable line. Not a confirmation, not an acceptance, not a booking, and only with a clean line test.

Before rung 0, once per environment

The ladder writes to one table and reads from five. Each of these must return **without an error**; zero rows is a pass.

```
SELECT ROWID FROM campaigns LIMIT 1;
SELECT announce_at, bidding_opens_at, bidding_closes_at, offers_at,
       decision_at, switch_window_at, reconcile_at FROM campaigns LIMIT 1;
SELECT ROWID FROM campaign_members LIMIT 1;
SELECT coverage_key, org_id, region, status FROM provider_coverage LIMIT 1;
SELECT acceptance_key, org_id, doc_type, doc_version FROM provider_terms LIMIT 1;
SELECT revision_key, bid_key, revision_no, payload FROM bid_revisions LIMIT 1;
```

And the partner has to be able to reach the cohort at all, which is coverage plus two admin decisions:

```
node scripts/cohort.mjs coverage <ORG_ID> --region "<REGION>"
```

```
UPDATE provider_orgs  SET approval_status = 'approved' WHERE org_id  = '<ORG_ID>';
UPDATE provider_users SET role = 'admin'               WHERE user_id = '<USER_ID>';
```

When a rung does not seem to land

WHAT YOU SEE	WHAT IT IS
Nothing moved on either side	The 60 second catalog memo. Wait, then reload.
Rung 2 ran and the desk still has no button	Correct. Rung 3 is what makes the stage `open`. The flag is not a stage.
Rung 3 ran and the desk still has no button	Coverage for that region is not `active` for that org, or the org is not `approved`, or the seat is `viewer`.
The bid is refused with "not open"	`site_config.bidding_enabled` is false, or rung 4 has already run.

WHAT YOU SEE	WHAT IT IS
"Bidding is not available right now"	`bid_revisions` or the fifteen extra `provider_bids` columns do not exist.
The bid is refused and nothing explains it	`provider_terms` does not exist. The terms gate fails closed, by design.
The member rail is not moving	That member has not joined this cohort, so the page polls every 120 seconds. Rung 0 was the only chance.
The cohort reads Decided far too early	`decision_at` is set. The partner stage reads it first, so a rung run out of order ends the run.
A rung was run twice	Harmless. It restamps the same date with a later minute.